

State migration design — PR1.4d through PR1.5+

A design pass undertaken before resuming implementation on the `per-voice-supervisors` branch. Captured 2026-05-08 from a session that audited `MIDIScheduler.purs` end-to-end and reshaped the design question accordingly.

This is the operational sequel to `per-voice-refactor-plan.md`, narrowed to the migration of state and dispatch responsibilities out of `MIDIScheduler`. The plan doc named directions ("`tidal_state` gen_server held by the supervisor"); this doc commits to specific shapes with reasoning, after reading the code rather than guessing at it.

0. Scope

In: the migration of every responsibility currently held by `Tidal.MIDIScheduler` into the per-voice supervision tree introduced by PR1.1–1.4c. Out: hot reload (PR2), cell-as-module (PR3), eDSL work (Path B from `live-coding-feasibility.md`).

The end state is that `MIDIScheduler.purs` is deleted. Every code path currently in it has a migration target named below.

1. Audit findings

These shape the design. Numbers reference `MIDIScheduler.purs` line numbers as of the start of this work.

1.1 State map

`MIDISchedulerState` (line 257-302) holds:

Field	Kind	Read sites at dispatch time
<code>config</code>	static	every event (look-ahead, cycle math, gate config)
<code>tracks</code>	runtime	tick walks all tracks
<code>continuousTracks</code>	runtime	tick samples all continuous
<code>bridgeClient</code>	socket	every MIDI emit
<code>oscClient</code>	socket	every gate/CV/ESX emit
<code>bindings</code>	registry	one site (compositional <code>#</code> , line 657)
<code>continuousBindings</code>	registry	only at install (<code>PlayByNameExpr</code>)
<code>sinkTypes</code>	registry	only at install (<code>PlayByName*</code> type-check)
<code>slots</code>	env	NO consumer found in the dispatch path
<code>midiDevices</code>	env	every MIDI emit (most-read shared map)
<code>fh2VoiceChannels</code>	env	every <code>Fh2TriggerTrack</code> event, every <code>Fh2Shape</code>

1.2 Five **ParsedTrack** variants

GateTrack	– sample-name pattern → gate (+ optional CV pre-set)
CVTrack	– numeric pattern → sustained /cv updates
ESXTrack	– numeric pattern → ESX-8CV slot
BoundTrack	– pattern + name + binding + params (#-joins)
Fh2TriggerTrack	– pattern + voice-id (channel resolved at fire)

GateTrack / **CVTrack** / **ESXTrack** are pre-Binding-era — they hard- code their destination. The Binding-era replacement is **BoundTrack** + a registered binding. Migration target for the three legacy variants: deprecate at the verb level (**UpdateGateTrack** / **UpdateCVTrack** / **UpdateESXTrack**) by mapping them to BoundTracks with synthesized bindings under reserved names (**__gate_<ch>**, **__cv_<bus>**, **__esx_<slot>**). User-visible behaviour unchanged.

1.3 The two kinds of **#**

Both live inside **BoundTrack** dispatch (line 561-675), filled at **PlayByName** time (line 886-891). They look identical at the parse layer (a **Map String (Pattern String)** of param name → pattern) but do different things at fire time:

(a) Slot overrides — line 604-614. For each PrimAction, if a param matches a known slot (currently only **vel** for MidiNote), sample the param pattern at the event's cycle and override the field on this PrimAction. Self-contained; no registry read.

(b) Compositional # — line 640-675. For each param NOT consumed as a slot override, look the name up in **state.bindings**. If it matches another registered binding, fire that binding's **MidiCC** actions with the joined value. **Only MidiCC is composed today** (line 675: **_ -> pure unit -- only MidiCC composition for now**). This is the cross-binding fanout.

Both kinds need: voice samples each param pattern at the event's cycle position, attaches to the event payload. Dispatcher decides which kind each is (slot override → consumed by named-action match; compositional → matches another binding name).

1.4 Continuous voices

Sampled once per tick (currently 50ms = 20Hz) using **samplePatternAt :: Rational -> Pattern Number -> Maybe Number** (line 1167-1180). NOT **queryArc** — different query type entirely.

Two destinations: **ContMidiCC { device, channel, cc }** and **ContCV { bus, transforms }**. Dispatched via **dispatchContValue** (line 1270-1292) with no latency adjustment, no event-time book-keeping.

Registered via **parseContBinding** (line 1241-1251) from **bind <name> midi-cc-cont ... / bind <name> cv-cont ...**. The track is installed when **PlayByNameExpr** evaluates to a **VNumPattern** and the name is in **continuousBindings**.

1.5 FH-2 — three verbs, two dispatch shapes

- **Fh2Envelope** (line 1056-1072) — registry write to **fh2VoiceChannels**; auto-registers **fh2** MIDI device alias.
- **UpdateFh2TriggerTrack** (line 1074-1087) — installs an **Fh2TriggerTrack**. Dispatch reads **fh2VoiceChannels** to resolve channel at fire time.
- **Fh2Shape** (line 1089-1118) — fires four CCs immediately (live ADSR), no track installed.

Fh2TriggerTrack is structurally a MIDI voice with one extra indirection: voice-id → channel via shared map. **Fh2Shape** is a fire-and-forget no-track verb.

1.6 Dormant / unclear

state.slots (line 293) is registered via **SetSlot** (line 1033- 1038) and serialized for the **state** verb (line 1519-1522), but I found no consumer in the dispatch path. Either: (a) dormant feature, (b) read site I missed, (c) future placeholder. **Flag for user to resolve before deletion or migration.**

1.7 State publication

publishState (line 1427-1430) runs every tick, serializes the full state to JSON, writes to **StateBus** ETS. Calypso reads from ETS. ~20 writes/sec; ETS insert is sub-microsecond.

After migration, no single process holds the full state. Snapshot must be assembled from voice_sup + dispatcher (and clock for tempo). Two viable models: (i) periodic publisher gen_server walks both; (ii) each owner publishes its slice, reader stitches at read time.

2. Design decisions

2.1 Where does each piece of state live

Decision: dispatcher-cohesive, with one exception.

Hot-path argument is decisive. State read by every event must be on the same process as the dispatcher; cross-process **gen_server:call** on a hot path is unacceptable. The fields read at every-event-rate (**midiDevices**, the local copy of **bindings** for the voice in question, **fh2VoiceChannels**) all live on the dispatcher. The plan doc's hand-wave at "preferred shape is a separate **tidal_state** gen_server" is reconsidered: a separate process buys SRP-cleanness at hot-path cost.

The exception is the **clock** — owns BPM and the tick. It exists already; no field migrates *from* the dispatcher *to* the clock.

Specifically:

Field	Lives on	Reason
config	clock + dispatcher	clock owns bpm/tickMs; dispatcher owns gateConfig
bindings	dispatcher	hot-path read for compositional #; install-time read for new voices
continuousBindings	dispatcher	install-time read; close to discrete bindings
sinkTypes	dispatcher	install-time read; derived from bindings so co-located

Field	Lives on	Reason
<code>midiDevices</code>	dispatcher	every MIDI emit reads it
<code>fh2VoiceChannels</code>	dispatcher	every FH-2 trigger reads it
<code>slots</code>	TBD	dormant; flag for user before designing
voice runtime state	per-voice gen_server	the whole point of the refactor

`publishState` (snapshot) is a separate concern handled in §2.6.

2.2 The `bind` verb routing

The `bind` verb mutates `bindings` (and `sinkTypes`). It does NOT install a voice on its own — it just registers a name. A subsequent `<name> <pat>` (`PlayByName`) installs the voice.

Today (post-PR1.4c): `bind` goes only to `MIDIScheduler`. Post-migration: `bind` goes only to dispatcher.

No dual-write needed: `bind` is registry-only. The earlier "dual-write to preserve `#` joins" plan was based on guessing that the registry was needed by a separate process for `#` joins. Now we know `#` joins read the registry from inside dispatcher dispatch — which is where the registry already lives in PR1.4c. The dispatcher is the single source of truth.

2.3 The `#` joins implementation

Voice computes enriched events. Dispatcher routes both kinds.

Voice state extends to hold `params :: Map String (Pattern String)`, populated from the WS handler at install. On each `compute_until`, voice queries pattern AND samples each param pattern at each event's cycle position. Event payload becomes:

```
{ token :: String
, wallTimeUs :: Number
, params :: Map String String -- pre-sampled per-event values
}
```

Dispatcher receives this, looks up binding by voice name. For each `PrimAction` in the binding:

- Walk `PrimAction` normally for the token.
- For each param in `event.params`:
 - If param matches a known slot for this `PrimAction` (e.g. `vel` for `MidiNote`), apply override.
 - Otherwise: look the param name up in `bindings`. If it matches another binding, fanout-fire that binding's `MidiCC` actions with the param's value.

This keeps the slot/composition decision on the dispatcher (where the registry lives). The voice doesn't need to know about other voices.

2.4 Continuous voices — same `gen_server`, polymorphic state

Per the link-spike-as-universal-clock simplification.

Voice state is a sum:

```
data VoiceKind
  = Discrete
    { pattern :: Maybe (Pattern String)
    , binding :: Binding
    , params :: Map String (Pattern String)
    , phase :: ...
    , lastEmittedUntil :: Number
    , muted :: Boolean
    }
  | Continuous
    { pattern :: Maybe (Pattern Number)
    , dest :: ContDest
    , muted :: Boolean
    }
```

Both subscribe to the same **Window** broadcast. **computeUntil** dispatches:

- **Discrete**: queries pattern over **[fromCycle, toCycle)**, produces scheduled events (current behaviour).
- **Continuous**: samples pattern at **currentCycle**, produces one immediate event.

Both push events to dispatcher. Dispatcher routes:

- Discrete event → dispatch via voice's binding (looked up by name).
- Continuous event → dispatch via voice's **ContDest** (looked up by name in **continuousBindings**).

Key implication: events from continuous voices need a flag (or a separate cast pattern) so the dispatcher knows to consult **continuousBindings**, not **bindings**. Two coherent options:

(i) Two cast patterns: **{event, ...}** for discrete, **{cont_event, ...}** for continuous. Dispatcher branches on the message type. **(ii)** Voice carries kind in its name registration (**add_voice** caller sets a **kind** flag); dispatcher's **set_binding** / **set_continuous_binding** respect this and the unified **dispatch_event** reads the correct map.

Pick (i). Cleaner: the message shape encodes the dispatch path, no per-name kind lookup needed. Dispatcher's **bindings** and **continuousBindings** maps are independently read.

2.5 FH-2 voices — new PrimAction variant

Decision: extend **PrimAction with a **Fh2Trigger** variant.**

Today's **Fh2TriggerTrack** is a **ParsedTrack** variant (line 233) with its own dispatch path. Migrating it as-is would mean voices need to know they're FH-2 voices, which complicates the polymorphic voice state. Instead, treat FH-2 trigger as a kind of MIDI emission: a **PrimAction** that captures **voice :: Int** and resolves **channel :: Int** at dispatch time.

```
data PrimAction
= ...
| Fh2Trigger { voice :: Int, defaultNote :: Int }
-- channel is resolved by the dispatcher via fh2VoiceChannels;
-- device alias is hardcoded "fh2"; durationMs is fixed at 200ms
-- (matches current behaviour).
```

The WS handler's `fh2-trigger <voice> <pattern>` verb installs a voice with binding `[Fh2Trigger { voice, defaultNote: 60 }]`. The voice is then a regular discrete voice; dispatcher's `Fh2Trigger` handler does the channel resolve.

`Fh2Envelope` and `Fh2Shape` stay as direct dispatcher casts (no voice involved):
`dispatcher:fh2_envelope(voice, output, channel)` and `dispatcher:fh2_shape(voice, a, d, s, r)`.

This unifies the voice model — every voice is either Discrete with some PrimActions, or Continuous with a ContDest. FH-2 is a Discrete voice with a specific PrimAction variant.

2.6 State publication

Decision: periodic publisher gen_server.

A dedicated `tidal_state_pub` gen_server ticks (e.g. every 100ms), calls `voice_sup:gather_state()`, calls `dispatcher:snapshot()`, calls `clock:snapshot()`, stitches the JSON, writes to StateBus. ~10 Hz publication is plenty for Calypso's `state` verb; the current 20 Hz is overkill (it's just whatever the scheduler tick happened to be).

Why a dedicated publisher rather than each owner publishing its slice:

- Calypso reads one ETS row, not three. Stitching at read time would push complexity into Calypso.
- Decouples publication rate from tick rate. Could go faster if a consumer needs it; could go slower if we want.

Why not just have the clock do it (since it already ticks):

- The clock should be lean. Adding "walk the whole supervision tree and serialize" couples it to a non-clock concern.

2.7 Slot bus

Decision: confirm with user before designing.

`state.slots` has a write path (`SetSlot` verb), is serialized in the JSON snapshot, but has no consumer in the dispatch code I audited. Three possibilities:

- **(a) Dormant** — registered but never wired up. Migration: drop the verb and the field.
- **(b) Read site missed** — keep grepping; design once the consumer is found.
- **(c) Future placeholder** — leave the API surface, migrate the field to dispatcher, no consumer until the future feature lands.

User to confirm. Likely (a) — the comment "Param/input slot env (modulation values)" suggests it was a sketch.

3. Migration path

3.1 PR1.4d — Voice + dispatcher own discrete dispatch end-to-end

Two commits:

PR1.4d-i — `bind` verb routes to dispatcher. Voice registration shape stabilizes.

- WS handler's `addBinding` message → `dispatcher:set_binding` (was: `MIDIScheduler`).
- WS handler's `removeBinding` → `dispatcher:remove_binding`.
- WS handler's `registerMidiDevice` → `dispatcher:register_midi_device`.
- `MIDIScheduler`'s `AddBinding` / `RemoveBinding` / `RegisterMidiDevice` handlers stay (still wired by `MIDIScheduler.spawn`) but receive no messages — dead code as of this commit. Removed in 1.4e.

PR1.4d-ii — `playByName` / `PlayByNameP` / `PlayByNameExpr` route to voices.

- WS handler reads binding from dispatcher, parses pattern + params, installs voice via `voice_sup:set_voice(name, kind, pattern, params, binding)`. (`set_voice` = upsert: replaces if voice for that name exists.)
- Voice state extends with `params :: Map String (Pattern String)`.
- Voice's `computeUntil` samples each param pattern at each event position; events carry pre-sampled `params :: Map String String`.
- Dispatcher's `dispatchEvent` consumes per-event params: applies slot overrides, performs compositional fanout via `bindings` registry lookup.

After 1.4d-ii: discrete voices fully run on the new tree. MIDI- Scheduler's `BoundTrack` dispatch is dead (no patterns reach it).

3.2 PR1.4e — Dismantle `MIDIScheduler`'s discrete path

- Remove `tracks` field, `BoundTrack` walk, `AddBinding` / `RemoveBinding` / `RegisterMidiDevice` handlers from `MIDIScheduler`.
- Migrate shared helpers (`noteNameMidi`, `interpretCV`, `clamp7bit`, `param7bit`, `voctValue`, `noteEntry`) into a new module `Tidal.Dispatch.Helpers`. Update Dispatcher's imports.
- `MIDIScheduler` shrinks to: clock+tick book-keeping, continuous voice tick, FH-2 verbs, legacy `GateTrack/CVTrack/ESXTrack` walks (still alive at this commit).

3.3 PR1.5 — Continuous voices migrate

- Voice state gains `Continuous` variant.
- WS handler's `PlayByNameExpr`: when name matches a `continuousBinding`, install via `voice_sup:set_voice(name, Continuous, ...)`.
- Voice's `computeUntil` for `Continuous` produces immediate-emit events.
- Dispatcher's `dispatch_cont_event` looks up `continuousBindings`, emits CC/CV.

After 1.5: continuous voices run on new tree. `MIDIScheduler`'s `continuousTracks` walk is dead.

3.4 PR1.6 — FH-2 migrates

- New `PrimAction.Fh2Trigger { voice, defaultNote }` constructor.
- WS handler's `fh2-trigger` verb installs a voice with that `PrimAction`.
- Dispatcher's `Fh2Trigger` handler resolves channel via `fh2VoiceChannels` and emits.
- `Fh2Envelope` / `Fh2Shape` route to dispatcher directly (no voice).

After 1.6: `MIDIScheduler.purs` is empty modulo helpers and the legacy `GateTrack/CVTrack/ESXTrack` code (which already migrated to synthesized `BoundTrack` equivalents at the verb-handling level).

3.5 PR1.7 — Legacy verbs synthesize `BoundTracks`

- WS handler's `UpdateGateTrack`, `UpdateCVTrack`, `UpdateESXTrack` install voices with auto-generated bindings under reserved names.
- `MIDIScheduler.purs` is deleted in this commit.

3.6 PR1.8 — State publisher

- `tidal_state_pub` `gen_server` reads from `voice_sup` + dispatcher + clock at 10Hz, writes JSON to `StateBus`.
- Calypso's `state` reading is unchanged (same ETS row, same JSON shape).

4. Failure modes and rollback strategy

This is a multi-PR refactor on a feature branch. Each numbered PR above is a coherent commit — the branch should compile and pass tests at each one. If a PR proves to be irreconcilable mid-flight:

1. **Revert the bad commit**, keep the prior state.
2. **Add a `failure-log` section to this doc** documenting what went wrong: what assumption broke, what alternative was tried, what the root cause was.
3. **Reset the branch** to before the problematic PR and adjust the design here before re-trying.

Common failure modes to anticipate:

- **PureScript Array vs Erlang array confusion** — already hit once in PR1.4b. Voice/dispatcher state with collection types needs to match the runtime encoding. Test in `erl -noshell` before committing each PR.
- **Newtype/sum encoding** — `purs-backend-erl` wraps every ADT constructor as a tuple. The `VoiceKind` sum type especially: each variant is a tagged tuple at runtime. Erlang code that deconstructs needs to know this.
- **Cast ordering** — voices push to dispatcher; dispatcher mutates state. If a `bind` verb reaches the dispatcher *after* the voice starts emitting events for that name, the dispatcher's binding lookup fails. Pre-PR1.4d this was bounded by `MIDIScheduler` being a single mailbox. Post-PR1.4d the dispatcher and `voice_sup` are parallel. Mitigation: WS handler does `dispatcher:set_binding` *before* `voice_sup:set_voice` (sequential calls, dispatcher's call is synchronous so it's strictly happens-before).
- **State publisher race** — `tidal_state_pub` reads from N processes at non-atomic moments. Acceptable per the plan doc's §5 ("`state` snapshot is eventually consistent"). Document this in the publisher's module comment.

See also:

- [docs/per-voice-refactor-plan.md](#) — upstream plan doc.
- [docs/live-coding-feasibility.md](#) — design exploration.
- [src/Tidal/MIDIScheduler.purs](#) — the file being dismantled.
- [src/Tidal/Dispatcher.purs](#) — the migration destination for dispatch logic.
- [src/Tidal/Voice.purs](#) — the migration destination for per-voice state.

5. Failure log

(empty — to be populated if we hit irreconcilable problems and have to reset)